

SocialLab

Arquitectura de datos masivos aplicada



Esto es lo que vais a montar →

[Github.com/pabloccanizares/SocialLab](https://github.com/pabloccanizares/SocialLab)



¿Qué es SocialLab?

Una red social pedagógica para aprender arquitectura de datos masivos

TWITTER FALSO PEDAGÓGICO

Simulamos una red social tipo Twitter para aprender a diseñar, limpiar y migrar datos a gran escala.



ESCENARIO NARRATIVO

Una red social cerró y os han contratado para **migrar** sus datos a una plataforma **nueva**.



2 600

usuarios



62 000

posts



120 000

likes



32 000

follows

ENTIDADES PRINCIPALES



User



Post



Like



Follow



Hashtag

PROBLEMAS HEREDADOS



Timestamps
en 5 formatos



Encoding
roto



Spam



Duplicados



Huérfanos

Qué vais a aprender

1



Bases de datos poliglotas

Mongo + Neo4j en el mismo sistema.

2



ETL Spark a escala

raw → silver → gold.

3



6 modelos ML

aplicados a datos de red social.

4



App real

cómo se conecta todo con FastAPI + frontend.

5



Bonus cloud gratis

Atlas + Aura + Databricks CE.



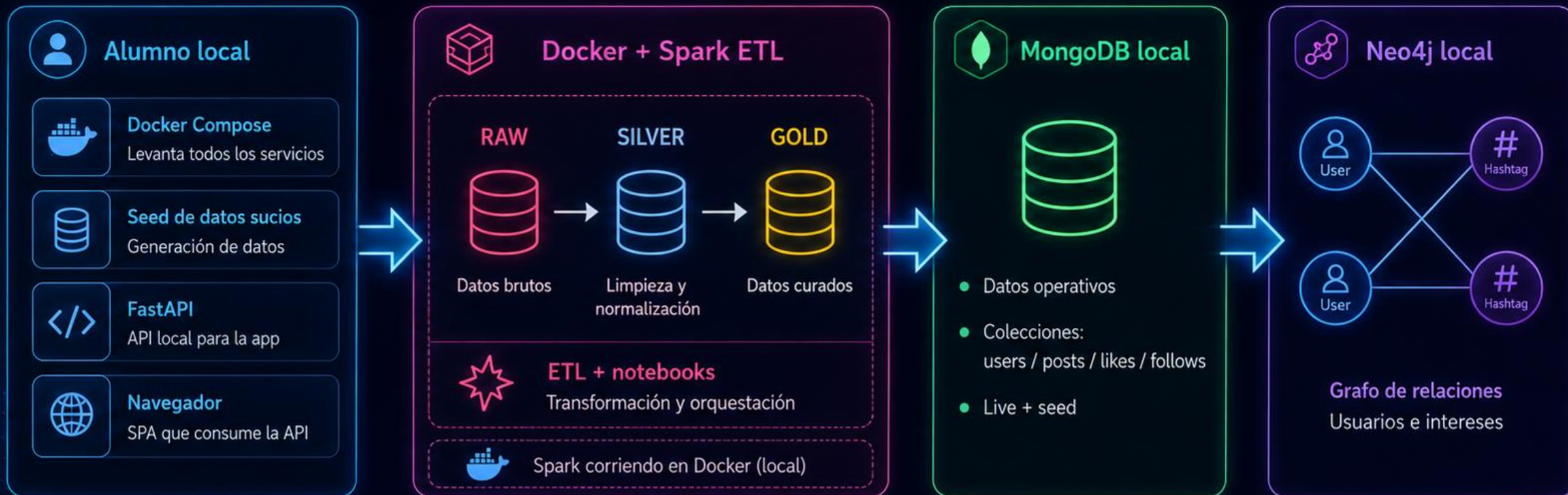
Cada bloque que vimos en clase tiene **un sitio en este sistema.**

Vista general

Arquitectura completa de SocialLab



Todo corre en local con Docker



1

Seed

Datos sucios



2

ETL

Limpieza y transformación



3

Carga

A MongoDB y Neo4j



4

App consulta

Explora y analiza

MongoDB • estado operativo



Rol: lo que la app lee y escribe en vivo.

Datos sociales en tiempo real para experiencias personalizadas.



Colecciones principales en uso



users

Perfiles y datos de usuarios



posts

Publicaciones del usuario



likes

Me gusta hacia publicaciones



follows

Relaciones de seguimiento



events

Eventos en vivo y actividades



user_stats

Estadísticas agregadas del usuario



hashtag_trends

Tendencias de hashtags en tiempo real



Campo clave destacado

origin: **seed** | **live**



¿Por qué Mongo?



Esquema flexible

Evoluciona con el producto sin migraciones complejas.



Escrituras rápidas

Alto rendimiento para cargas y eventos en vivo.



Sin joins complejos

Datos embebidos y relaciones simples por diseño.

</> Documento de ejemplo: users (vista JSON)

```
1 {
2   "_id": ObjectId("664c8f9e8b6d4b2f9a2e1c01"),
3   "username": "auroradev",
4   "display_name": "Aurora Dev",
5   "email": "aurora.dev@example.com",
6   "bio": "Construyendo comunidad e impacto social 🚀",
7   "avatar_url": "https://cdn.sociallab.app/avatars/aurora.png",
8   "created_at": ISODate("2024-05-18T12:34:56Z"),
9   "origin": "live",
10  "profile": {
11    "location": "Madrid, España",
12    "website": "https://aurora.dev",
13    "join_source": "web"
14  },
15  "stats": {
16    "posts_count": 42,
17    "followers_count": 1287,
18    "following_count": 356,
19    "likes_received": 19876
20  },
21  "preferences": {
22    "notifications": true,
23    "language": "es",
24    "theme": "dark"
25  }
26 }
```

● **seed** = datos iniciales / carga base | ● **live** = datos en vivo / generados por la app



La app vive sobre **MongoDB** en tiempo real

Neo4j • grafo social



Rol: relaciones entre usuarios y temas.



Nodos: User, Hashtag.



Relaciones: (:User)-[:FOLLOWS]->(:User)
(:User)-[:INTERESTED_IN]->(:Hashtag)



:User

:Hashtag

Ejemplos de consultas (Cypher)

// 1) Amigos de amigos (friends-of-friends)

```
MATCH (u:User {id: $userId})-[:FOLLOWS*2]-(f:User)
WHERE u <> f AND NOT (u)-[:FOLLOWS]-(f)
RETURN f, count(*) AS caminos
ORDER BY caminos DESC LIMIT 20;
```

// 2) Hashtags en los que está interesado un usuario

```
MATCH (u:User {id: $userId})-[:INTERESTED_IN]->(h:Hashtag)
RETURN h, count(*) AS peso
ORDER BY peso DESC;
```

// 3) Comunidades (aprovechando algoritmos de grafo)

```
CALL gds.louvain.stream('social', { relationshipWeightProperty: null })
YIELD nodeId, communityId
RETURN gds.util.asNode(nodeId).id AS user, communityId
ORDER BY communityId, user;
```



Queries que NO se hacen bien en Mongo

✗ shortest-path

✗ comunidades

✗ friends-of-friends



¿Por qué Neo4j?

Las consultas de grafo son aquí
mucho más rápidas.



Data Lake + Spark · ETL

De datos crudos a información lista para consumir



IDEA CLAVE

raw es materia prima, **gold** es lo servible.

El mismo código, dos formas de ejecutarse



Local (PySpark)

- ✓ Desarrollo y pruebas rápidas
- ✓ Ejecución en tu máquina
- ✓ PySpark en modo local



Databricks (Notebooks)

- ✓ Ejecución distribuida y escalable
- ✓ Notebooks colaborativos
- ✓ Mismo código, mayor potencia

FastAPI + frontend

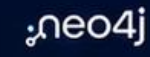
 **Backend:** Python · async (motor + neo4j driver)

 **Frontend:** SPA vanilla JS con Cytoscape.js para visualizar grafos

Backend (Python · async)

 FastAPI

 motor
MongoDB Async
Driver

 neo4j
Neo4j
Driver

1 feed Twitter

¿Qué está pasando?

Buscar



María López @mlopez · 5m

Increible lo que está pasando con la ciencia de datos 🤖
Los datos cuentan historias.

12 34 128



DataGeek @datageek · 12m

Nuevo dataset público sobre interacciones sociales.
Listo para analizar en Neo4j + Spark 🚀

8 21 67



Nina @nn_analytics · 25m

Graficando relaciones con @Cytoscape.js
Es increíblemente potente.

3 11 42

2 Neo4j (grafo social)

Buscar nodo...



● Usuario
● Hashtag
● Mención
● Enlace
♥ Retweet

3 Spark/ML (modelos)

Modelos y Métricas

Modelo activo: spam_detector_v2

Precisión

0.92

Recall

0.89

F1 Score

0.90

AUC

0.95

Predicciones recientes

texto	predicción	score
Gana dinero rápido ahora!!!	SPAM	0.98
Nueva investigación sobre IA y datos.	HAM	0.12
Haz clic aquí para reclamar premio	SPAM	0.95
Conferencia de grafos y redes sociales	HAM	0.08
Oferta limitada, actúa ya!!!	SPAM	0.93

Distribución de clases



Top features (IMPORTANCIA)



Idea clave:

Si el ETL está mal, la app muestra **basura**.

Datos sucios intencionados

El ETL existe para arreglar esto

1



2024-05-17T12:30:00Z

1715945400

05/17/2024 12:30 PM

17/05/2024 12:30

hace 2h

5 formatos de timestamp
ISO, epoch, US, EU, "hace 2h"

2



8%

usuarios duplicados
mismos usuarios, múltiples IDs

3



4%

campos faltantes
datos incompletos o nulos

4



#DataBricks

databricks

DATABRICKS

data bricks

Hashtags inconsistentes
con/sin #, mayúsculas, espacios

5



Encoding roto
á → Ã±, ñ → Ñ±, etc.

6



5%

usuarios spam
+ ráfaga de 100 posts iguales

7



3%

likes/follows huérfanos
referencias sin origen válido

8



2%

self-follows
usuarios siguiéndose a sí mismos



Nadie va a tener datos limpios en su carrera

Pipeline ETL

De raw a productos analíticos y de grafo

1 RAW → SILVER

Limpieza y estandarización



Parsear timestamps

Normalizar formatos a UTC



Dedup por email

Eliminar duplicados por email



Fix encoding

Corregir codificación de texto



Filtrar huérfanos

Eliminar referencias sin dueño



Marcar spam

Detección y etiquetado de spam

Transforma

2 SILVER → GOLD

Enriquecimiento y modelado



Agregados

user_stats, hashtag_trends,
post_rankings



Features ML

Características listas
para modelos



Edges Neo4j

Relaciones (FOLLOW, INTERESTED_IN)
listas para grafo

Carga

3 FINAL

Publicación de productos



MongoDB Atlas

Colecciones analíticas
(agregados y docs)



Neo4j AuraDB

Nodos y relaciones
del grafo social



Cargado desde Spark

Jobs orquestados y reproducibles

</> Código clave



src/spark/etl_silver.py

Transformaciones RAW → SILVER



src/spark/etl_gold.py

Transformaciones SILVER → GOLD



La lógica vive en el ETL

Reproducible • Versionable • Trazable



Datos confiables



Productos reutilizables



Pipeline reproducible



Gobernanza y calidad

6 modelos ML en 3 bloques



supervised



spam_detector



engagement_predictor



virality_classifier



churn_predictor



ojo: data leakage a propósito

Pipelines • train/test • AUC • R^2



unsupervised



user_clustering

KMeans + silhouette



graph_ml



follow_recommender

Scoring híbrido: grafo + contenido



3 bloques • 6 modelos • una misma app



Modo scaffold vs solución



scaffold

El alumno ve el ejercicio con los algoritmos vacíos.

neo4j_basic_ex.py

```
1 from fastapi import APIRouter, Depends
2 from neo4j import AsyncSession
3
4 router = APIRouter()
5
6 @router.get("/users_by_hashtag")
7 async def users_by_hashtag(
8     session: AsyncSession, hashtag: str
9 ):
10     """
11     Devuelve los usuarios que han usado el hashtag
12     indicado y el número de veces.
13     Implementa la consulta Cypher.
14     """
15     raise NotImplementedError
```



solución

El profesor puede destapar la solución tras la clase.

neo4j_basic.py

```
1 from fastapi import APIRouter, Depends
2 from neo4j import AsyncSession
3
4 router = APIRouter()
5
6 @router.get("/users_by_hashtag")
7 async def users_by_hashtag(
8     session: AsyncSession, hashtag: str
9 ):
10     result = await session.run(
11         """MATCH (u:User)-[:USES]->(p:Post)-[:HAS]->(h:Hashtag)
12         WHERE toLower(h.name) = toLower($tag)
13         RETURN u.username AS username, count(*) AS total
14         ORDER BY total DESC""",
15         tag=hashtag
16     )
17     return [record.data() async for record in result]
```



La app arranca con **algoritmos en blanco**.



Según resolvéis ejercicios, las tarjetas del frontend **se encienden solas**.



Después de clase, el profesor puede **destapar la solución**.

Ejercicio 3

Top usuarios por hashtag

API Neo4j Cypher

0%



Ejercicio 3

Top usuarios por hashtag

API

Neo4j

Cypher

100%



Bloques de ejercicios

Neo4j — 12 endpoints en 3 bloques

1



basic (3)

- stats · influencers · communities

`MATCH simple + count`

2



intermediate (5)

- bridges · mutual-interests · hashtag-graph · my-communities · community-overlap

`multi-MATCH + collect`

3



advanced (4)

- shortest-path · ego-network · reach · famous-distances

`shortestPath + [*1..n]`

ML — 6 modelos en 3 bloques

1



supervised

Modelos que aprenden con datos etiquetados.



2



unsupervised

Descubren patrones sin etiquetas.



3



graph_ml

Modelos que entienden y aprenden sobre grafos.



La dificultad sube dentro de cada bloque



Empezáis por **basic**

“Destapar” bloques en clase

```
> ./lab.sh unlock neo4j basic |
```

1

Profesor ejecuta el comando



```
> ./lab.sh  
unlock neo4j basic
```

2

En ~3 s se reinicia
el contenedor app

app



reiniciando...



app



listo



3

Refrescáis el navegador
y veis el código bueno

<http://localhost:3000>

¡Bloque desbloqueado!

Funcionalidad habilitada



Próxima clase: siguiente bloque



Vosotros no tenéis que hacer nada. **Solo seguir.**

3 maneras de arrancar

1



Nativo

```
./start.sh
```



RAM

~1 GB



Para quien ya tiene
Mongo + Neo4j local



RECOMENDADO

2



Docker local

```
./lab.sh up
```



RAM

~3 GB ETL / 1 GB idle



La opción recomendada
para la mayoría

3



Cloud free

```
./lab.sh cloud
```



RAM

~150 MB



Para portátiles justos
o sin Docker



Elige según recursos y comodidad

Cloud 100% gratis

Tres servicios **free** para levantar SocialLab sin tarjeta de crédito

1



**MongoDB
Atlas M0**

- Free forever · 512 MB
- Base documental
- Ideal para datos operativos

2



**Neo4j
AuraDB Free**

- 200k nodos / 400k relaciones
- Grafo social
- Consultas Cypher y relaciones

3



**Databricks
Community Edition**

- Single-node
- Spark + notebooks
- Sin tarjeta de crédito

**0 €****setup ~22 min****sin tarjeta de crédito**

Guía paso a paso:
[docs/MIGRACION_CLOUD.md](#)

Tu app local



SocialLab

MongoDB
Atlas M0



Datos operativos

Neo4j
AuraDB Free



Grafo social

Databricks
Community Edition



ETL · Analítica · ML



Opción ideal para **portátiles modestos** o **sin Docker**

Comandos clave de lab.sh

No hace falta memorizar nada: `./lab.sh help` los lista todos

```
lab@workshop:~$  
  
./lab.sh up           # arranca todo (Docker)  
./lab.sh seed         # genera datos sucios  
./lab.sh etl          # corre el pipeline Spark  
./lab.sh train        # entrena modelos ML  
./lab.sh status       # qué bloques están desbloqueados  
./lab.sh cloud        # modo cloud free  
./lab.sh reset        # borra todo y empieza de cero  
  
>_ █
```



Comando comodín:
`./lab.sh help`



setup

up / cloud

para arrancar



datos

seed / reset

para preparar
y limpiar



ejecución

etl / train / status

para ejecutar y
ver el progreso



La idea es simplificar la práctica: **pocos comandos, flujo claro.**



Demo en vivo

Esto vende el sistema mejor que cualquier slide



Una pequeña demo conecta Docker, app, scaffold y resultados

Entrega y evaluación

Qué entregar, cómo se corrige y cómo cerrar la práctica

1



Qué entregar

Repo con vuestros
`_ex.py` rellenos

2



Bloques Neo4j (X %)

Correctness de las
12 queries Cypher

3



Bloques ML (X %)

Que los 6 modelos
entrenen y la UI
muestre métricas
razonables

4



Notebook prácticas (X %)

Ejercicios del
notebook



Bonus (X %): mejoras propias, optimización, modelos extra



Plazos



canal de dudas



tutoría



Preguntas

Dejad ~10 min para dudas